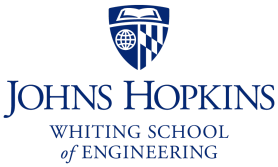
© Guven

Model Regularization




```
"Simplicity is the ultimate
sophistication." Leonardo da Vinci
```

In machine learning, **the most important aspect** of model development is **minimizing overfit** during training a model.

Question: *How do we really measure overfit?*

When a model *overfits* to the training dataset, the developed model may not perform strongly in practice. Though it might have achieved a high accuracy with the testing dataset, the model tried hard to fit to the noise and outliers in the training dataset.

A second reason why the model would not perform strongly in practice is that simply the new, never-seen-before test data was not known and did not exist in the training dataset. Thus, they were never learned in the first place.

Our concern about model overfitting is as in the former, and possible remedies are:

- Ensemble learning
- Cleaning/pre-processing the input data to remove outliers, noise, and use fewer features/dimensions in the model
  - Early stopping of the model training
- Regularization
- Abstraction

Regularization

Recall that we built a set of "simpler" Naive Bayes (NB) models in the ensemble learning module using a subset of the input dataset features. A set of M models was trained (each not using one of the features) and their outputs were weight-majority voted (using the target probabilities). The idea was that when we use **fewer features**, the model would be **simpler**. Now consider an analogy where we attempt to build a **simpler** Neural Network (NN) by applying two strategies:

1. Penalizing the sum of weights of layers, *aka* regularization
2. Dropping some network vertices randomly, *aka* drop-out ratio

The idea behind adding the L1 or L2 norm of weights to the cost function forces the optimizer to use smaller weights for the next layer output. Recall the perceptron equation: $y = w^\top x + b$

Penalizing weights is, in effect, similar to the NB exercise, pushing the training to skip some features or use fewer features.

NN **regularization** can be realized by an additional cost term based on the weights in the cost function optimized by Gradient Descent (GD) at each input batch to the NN training.

L1: $\lambda \|w\|_1 = \lambda \sum_{j=1}^{m} |w_j|$

L2: $\lambda \|w\|_2^2 = \lambda \sum_{j=1}^{m} w_j^2$

The hyperparameter λ is the regularization parameter. L2 regularization is referred to as **weight decay** since this cost forces the weights to diminish towards zero. When there are important features that explain the data (i.e., classification error is diminished), those weights will remain higher, but other features' weights are suppressed by the optimized cost function. In a way we lose some features by having their weights as 0 in the NN with regularization.

- Some Machine Learning developers consider L1 more robust than L2 (robustness means resistant to outliers in the data).
- L1 generates more unstable solutions than L2 (remember that NN training solves an optimization problem with Gradient Descent).
- L2 cost function is a convex function, but L1 is not (note that convex functions have unique global optimums).

Due to these reasons, L2 is more popular.

Referring to the Multilayer Perceptron Jupyter notebook, the L2 term can be added to the cost in the `compute_cost` function, such as:
`grad_w_h += 0.01 * sum(abs(self.w_h))`

Bias and Variance Tradeoff

Bias is the difference between the average prediction of the model and the ground truth. Variance is the variability of the model prediction. Models with high variance probably are overfit and do not generalize well. Mathematically, the following is the expected value of the classification error when y is predicted with a ground truth of t and an input test data point x,

$$\mathbf{E}\left[(y-t)^2) \mid x\right] \triangleq (y-y_\star)^2 + \mathrm{Var}[t \mid x]$$

where $y_\star = \mathbf{E}\left[t \mid x\right]$ is the best possible prediction. Decomposing further,

$$\mathbf{E}\left[(y-t)^2)\right] = \underset{bias}{(y_\star - \mathbf{E}\left[y\right])^2} + \underset{variance}{\mathrm{Var}(y)} + \underset{Bayes\ error}{\mathrm{Var}(t)}$$

We seek a higher bias to generate robust models to achieve lower variance.

Decision Tree Pruning

Decision tree post-pruning is the process of reducing the complexity of the raised tree by removing subtrees that carry less information in the model. Two major approaches are,

- Subtree replacement - Each node is replaced with its most popular class and tested for accuracy
- Subtree raising - A lower node is raised to cover for its parent

The below example shows two trees generated on the Iris dataset without and with pruning, and a simpler (more abstract) tree is preferred over the complex one.

Note that the `min_samples_leaf` parameter can also achieve direct pruning.

In [1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 72
import numpy as np
```

In [2]:
```
from sklearn.datasets import load_iris
from sklearn import tree

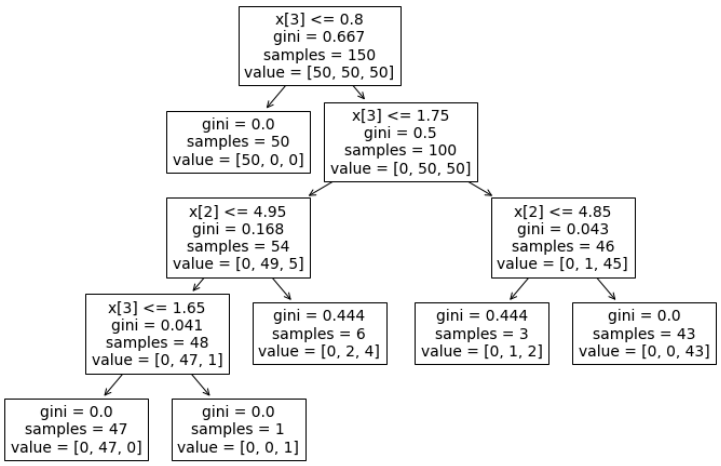
# Alpha beta pruning constant, default is 0
Alpha = 0.1

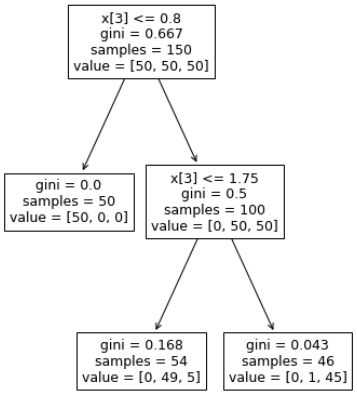
iris = load_iris()
X, y = iris.data, iris.target

clf1 = tree.DecisionTreeClassifier(random_state=0, min_samples_split=10).f
clf2 = tree.DecisionTreeClassifier(random_state=0, ccp_alpha=Alpha).fit(X,

_, ax = plt.subplots(nrows=1, ncols=2, figsize=(20, 8), gridspec_kw={'widt
tree.plot_tree(clf1, ax=ax[0])
tree.plot_tree(clf2, ax=ax[1])

plt.show()
```





In [3]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_r, X_s, y_r, y_s = train_test_split(X, y, test_size=0.5, stratify=y, ran

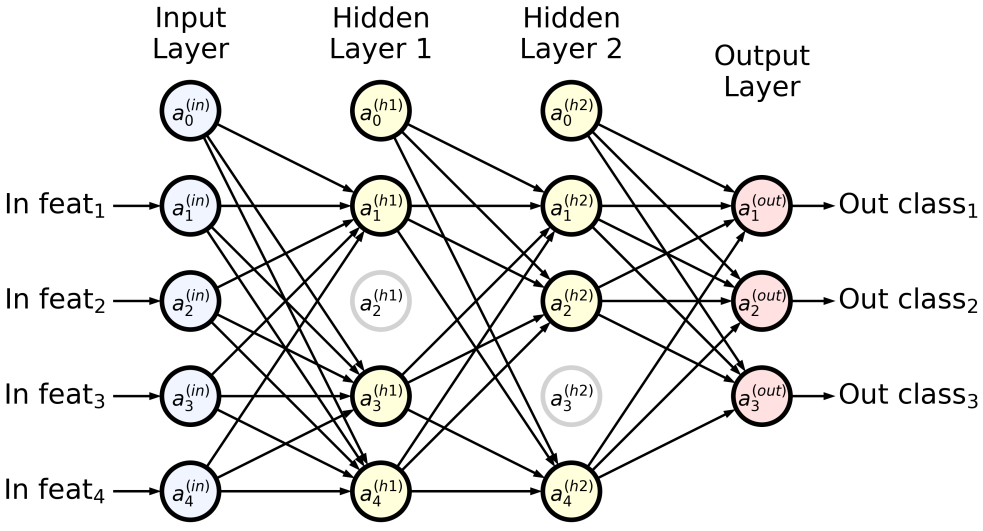
clf1 = tree.DecisionTreeClassifier(random_state=0, min_samples_split=10).f
clf2 = tree.DecisionTreeClassifier(random_state=0, ccp_alpha=Alpha).fit(X_

print(f'DT1 accuracy= {accuracy_score(y_s, clf1.predict(X_s)):.2f}')
print(f'DT2 (pruned) accuracy= {accuracy_score(y_s, clf2.predict(X_s)):.2f
```

```
DT1 accuracy= 0.89
DT2 (pruned) accuracy= 0.93
```

Neural Network Dropout

Simply, dropping neural network nodes can help reduce



overfitting. Ref: Srivastava, Nitish, et al. "Dropout: a simple way to prevent

neural networks from overfitting." The Journal of Machine Learning Research 15.1 (2014): 1929-1958.

The following NN shows $a_2^{(h1)}$ and $a_3^{(h2)}$ nodes as not used, dropped. Normally, dropout is applied randomly to hidden layers during training. A random dropout is applied at each batch. The effect of random dropout forces the NN to learn models redundantly since a single node cannot be reliably counted on.

It is expected the training take a longer time when dropout is used.

The dropout feature is unavailable with the `scikit-learn` library `MLPClassifier` function.

Tensorflow has the dropout feature through `h1_d = tf.nn.dropout(h1, dropout_p)` function.

Overfit Demonstration

Consider the following three SVM classifiers for a dramatic representation of overfitting. The first and last classifiers are either too general or too overfitted. Despite making more mistakes than the third, the second classifier is a better model due to its higher generalization ability.

In the code below, try different schemes by changing the input data points $X \in \mathbb{R}^{12\times 2}$ and 2 classes and see how the decision boundary changes. Note that the second and the third classifiers are RBF kernels with gamma set to 0.05 and 0.5, respectively. The gamma parameter has control over the *wiggling* of the decision boundary set by the RBF kernel.

In [4]:
```
# Training data, 12 data points
X_tr = np.array([
    [15, 11, 10.5, 12, 11.5, 15,      11.5, 14, 14.5, 12.5, 16, 17],
    [10, 11, 13, 14, 16, 18,          12, 13, 15, 16.2, 11.5, 14]]).T

y_tr = np.array([0,0,0,0,0,0,         1,1,1,1,1,1])
```

In [5]:
```
from sklearn import svm

# For the nicer plot
def get_minmax(_m, _X):
    return _X[:,0].min()-_m, _X[:,0].max()+_m, _X[:,1].min()-_m, _X[:,1].m

# Draw the linear decision boundary of an SVC easily
def svc_linear(_X, _y, _ax, C=10, color_db='r'):
    clf = svm.SVC(kernel='linear', C=C).fit(_X, _y)
    # dividing hyperplane
```

```
    a = -clf.coef_[0][0] / clf.coef_[0][1]
    x1_min, x1_max, _, _ = get_minmax(1, _X)
    x1_plot = np.linspace(x1_min, x1_max)
    x2_plot = a*x1_plot - clf.intercept_[0]/clf.coef_[0][1]
    _ax.plot(x1_plot, x2_plot, linestyle='--', color=color_db, linewidth=2
    return clf
```

In [6]:
```
def accuracy(_clf, _X, _y):  # percent
    return ((np.sum(_y==_clf.predict(_X))).astype(float)/_y.shape[0])*100

# We will find the decision boundary with a mesh plot
def svc_rbf(_X, _y, _ax, _gamma, C=10, color_db='r'):
    clf = svm.SVC(kernel='rbf', gamma=_gamma, C=C).fit(_X, _y)
    plot_decisionboundary(_ax, 0.01, _X, clf, color_db)
    return clf

# Plot the decision boundary of a classifier using mesh and contour, 2-dim
def plot_decisionboundary(_ax, _h, _X, _clf, color_db='r'):  # h = step si
    x1_min, x1_max, x2_min, x2_max = get_minmax(1, _X)
    xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, _h), np.arange(x2_min
    Y = _clf.predict(np.c_[xx1.ravel(), xx2.ravel()]).reshape(xx1.shape)
    _ax.contour(xx1, xx2, Y, colors=color_db, linestyles='dashed')
```

In [7]:
```
# Plot the data points
def plotX(_ax, _X, _y, _x1lim, _x2lim, mrk1='o', mrk2='+', col1='darkorchi
    _ax.scatter(_X[_y==0,0], _X[_y==0,1], marker=mrk1, color=col1, s=40)
    _ax.scatter(_X[_y==1,0], _X[_y==1,1], marker=mrk2, color=col2, s=70)
    _ax.get_xaxis().set_ticks([])
    _ax.get_yaxis().set_ticks([])
    _ax.set_xlim(_x1lim)
    _ax.set_ylim(_x2lim)
    _ax.set_xlabel(r'$x_1$', horizontalalignment='right', x=1.0)
    _ax.set_ylabel(r'$x_2$', horizontalalignment='right', y=1.0)
```

In [8]:
```
x1lim, x2lim = (X_tr[:,0].min()-1, X_tr[:,0].max()+1), (X_tr[:,1].min()-1,

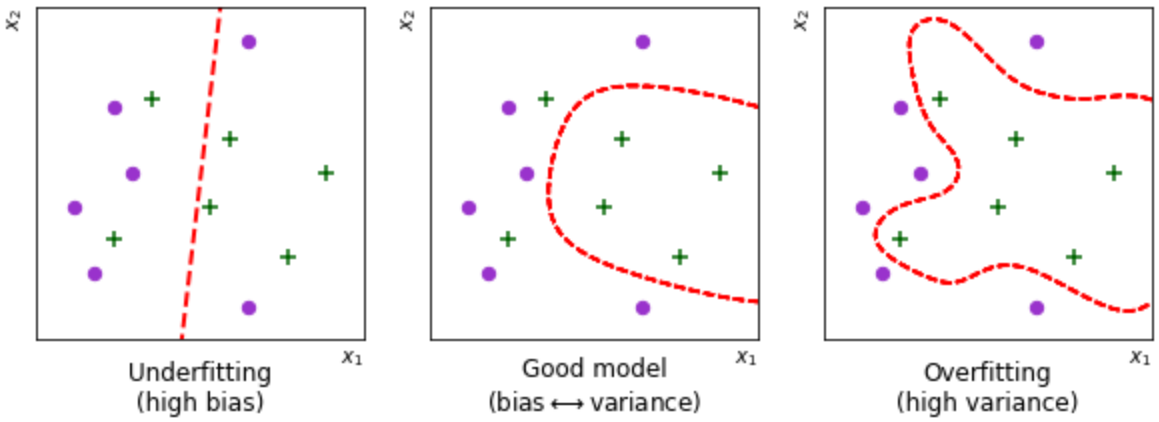
_, ax = plt.subplots(nrows=1, ncols=3, sharey='all', figsize=(10, 3), dpi=

plotX(ax[0], X_tr, y_tr, x1lim, x2lim)
svc_linear(X_tr, y_tr, ax[0])
ax[0].set_title('Underfitting\n(high bias)', y=-0.25)

plotX(ax[1], X_tr, y_tr, x1lim, x2lim)
svc_rbf(X_tr, y_tr, ax[1], 0.05)
ax[1].set_title('Good model\n(bias$\longleftrightarrow$variance)', y=-0.25

plotX(ax[2], X_tr, y_tr, x1lim, x2lim)
svc_rbf(X_tr, y_tr, ax[2], 0.5)
ax[2].set_title('Overfitting\n(high variance)', y=-0.25)

plt.show()
```



Note that these decision curves are generated by passing $[x_1, x_2]$ data points as a mesh with a granularity of 0.01. All data points are then checked for prediction by the model and a contour is generated for the outcomes y (which are either 0 or 1).

Never-seen-before Data Points

Now, let's see training re-classification and testing performances. A test of 4 data points was added, marked with the star `*` (ground truth circle class) and cross `x` (ground truth plus class) in red below.

If the model is general enough, then the new data points can be classified correctly.

In [9]:
```
# Add test data points to the dataset
X_test = np.array([
    [13, 12,      13, 13.3],
    [12.5, 13,    14, 13.7]]).T

y_test = np.array([0, 0, 1, 1])
```

In [10]:
```
fig = plt.figure(figsize=(5, 3), dpi=72)
ax = fig.add_subplot(111)

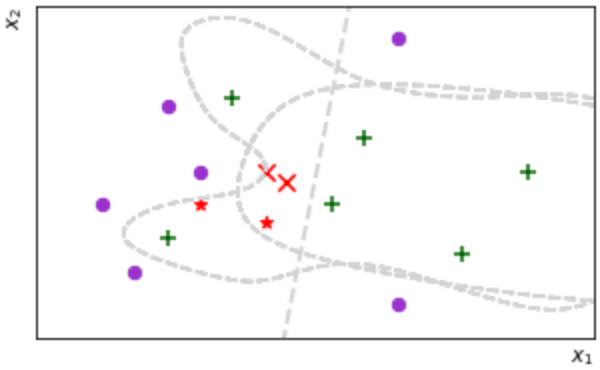
plotX(ax, X_tr, y_tr, x1lim, x2lim)
plotX(ax, X_test, y_test, x1lim, x2lim, mrk1='*', mrk2='x', col1='r', col2

clf1 = svc_linear(X_tr, y_tr, ax, color_db='lightgray')
clf2 = svc_rbf(X_tr, y_tr, ax, 0.05, color_db='lightgray')
clf3 = svc_rbf(X_tr, y_tr, ax, 0.5, color_db='lightgray')

plt.show()

print(f'SVC 1 Reclassification Acc= {accuracy(clf1, X_tr, y_tr):.1f}%')
print(f'SVC 2 Reclassification Acc= {accuracy(clf2, X_tr, y_tr):.1f}%')
print(f'SVC 3 Reclassification Acc= {accuracy(clf3, X_tr, y_tr):.1f}%')
print(f'SVC 1 Test Acc= {accuracy(clf1, X_test, y_test):.1f}%')
```

```
print(f'SVC 2 Test Acc= {accuracy(clf2, X_test, y_test):.1f}%')
print(f'SVC 3 Test Acc= {accuracy(clf3, X_test, y_test):.1f}%')
```




```
SVC 1 Reclassification Acc= 66.7%
SVC 2 Reclassification Acc= 83.3%
SVC 3 Reclassification Acc= 100.0%
SVC 1 Test Acc= 50.0%
SVC 2 Test Acc= 75.0%
SVC 3 Test Acc= 50.0%
```

In the plot above, 4 data points marked in color red constitute the test dataset. The test data is manually created, but then can you disagree with those 4 data points? The locations of those data points can easily happen in practice. As a result, the better model generalization achieves the better test performance. Let's also check their reclassification performances.

Interesting! The more general model has a higher test performance and a medium reclassification performance.

Let's see an example of a Multilayer Perceptron classifier with L2 regularization. The hyperparameter `alpha` controls L2 regularization for the library function `MLPClassifier`.

Note that scaling is applied to the MLP classifier. An MLP can also be trained to achieve 100% test accuracy.

In [11]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier

X_tr_sc = StandardScaler().fit_transform(X_tr)
X_ts_sc = StandardScaler().fit_transform(X_test)

# Update plot x1, x2 limits
x1lim2, x2lim2 = (X_tr_sc[:,0].min()-1, X_tr_sc[:,0].max()+1), (X_tr_sc[:,


def plotNN(_X_tr, _y_tr, _X_ts, _y_ts, _x1lim, _x2lim, _mlp1, _mlp2, _t1,
    plt.figure(figsize=(12, 2.5), dpi=72)

    ax=plt.subplot(1, 4, 1)
```

```
    plotX(ax, _X_tr, _y_tr, _x1lim, _x2lim)
    plot_decisionboundary(ax, 0.01, _X_tr, _mlp1, 'pink')
    plt.title(_t1 + ' (trained)', fontsize='medium')

    ax=plt.subplot(1, 4, 2)
    plotX(ax, _X_ts, _y_ts, _x1lim, _x2lim, mrk1='*', mrk2='x', col1='r',
    plot_decisionboundary(ax, 0.01, _X_tr, _mlp1, 'pink')
    plt.title(_t1 + ' (test)', fontsize='medium')

    ax=plt.subplot(1, 4, 3)
    plotX(ax, _X_tr, _y_tr, _x1lim, _x2lim)
    plot_decisionboundary(ax, 0.01, _X_tr, _mlp2, 'gold')
    plt.title(_t2 + ' (trained)', fontsize='medium')

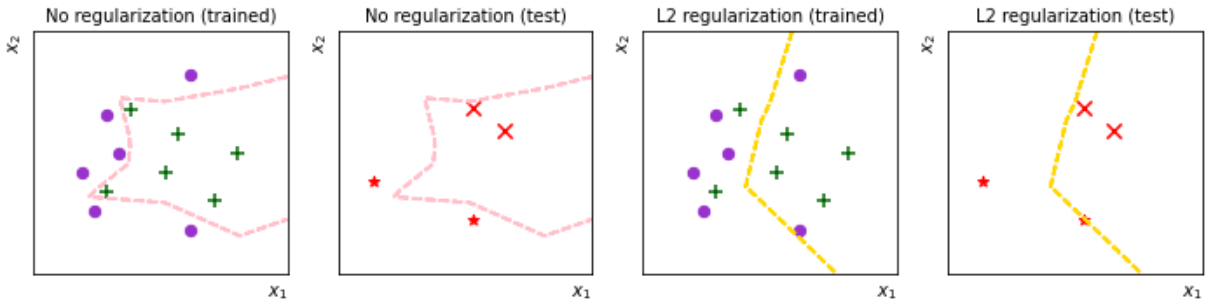
    ax=plt.subplot(1, 4, 4)
    plotX(ax, _X_ts, _y_ts, _x1lim, _x2lim, mrk1='*', mrk2='x', col1='r',
    plot_decisionboundary(ax, 0.01, _X_tr, _mlp2, 'gold')
    plt.title(_t2 + ' (test)', fontsize='medium')
    plt.show()

    print(f'MLP 1 Reclassification Acc= {accuracy(_mlp1, _X_tr, _y_tr):.1f
    print(f'MLP 1 Test Acc= {accuracy(_mlp1, _X_ts, _y_ts):.1f}%')
    print(f'MLP 2 Reclassification Acc= {accuracy(_mlp2, _X_tr, _y_tr):.1f
    print(f'MLP 2 Test Acc= {accuracy(_mlp2, _X_ts, _y_ts):.1f}%')
```

In [12]:
```
mlp1 = MLPClassifier(hidden_layer_sizes=(10,), alpha=0, random_state=0, ma
mlp2 = MLPClassifier(hidden_layer_sizes=(10,), alpha=0.5, random_state=0,

# mlp1 = MLPClassifier(hidden_layer_sizes=(60,20,5,), alpha=0, random_stat
# mlp2 = MLPClassifier(hidden_layer_sizes=(60,20,5,), alpha=0.5, random_st

plotNN(X_tr_sc, y_tr, X_ts_sc, y_test, x1lim2, x2lim2, mlp1, mlp2, 'No reg
```




```
MLP 1 Reclassification Acc= 100.0%
MLP 1 Test Acc= 100.0%
MLP 2 Reclassification Acc= 66.7%
MLP 2 Test Acc= 100.0%
```

Question: Which model do you prefer, left or right?

PyTorch Dropout Demonstration

PyTorch is a popular Python-based computational graph framework that implements deep learning algorithms. The following code is a typical neural network class based on `nn.Sequential`. Note that dropout requires a longer training session.

Windows 10 (x64)

Use a separate Anaconda environment for PyTorch:
`conda create -n torch -c pytorch pytorch torchvision torchaudio cpuonly jupyter matplotlib pandas scikit-learn` ↩
`conda activate torch` ↩

Run this notebook under the virtual environment `torch`.

PyTorch API:

https://pytorch.org/docs/stable/index.html

In [13]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

import torch

print(f'PyTorch version= {torch.__version__}')
print(f'CUDA available= {torch.cuda.is_available()}')  # CUDA optional - T
```

```
PyTorch version= 2.1.2
CUDA available= True
```

In [14]:
```
class PyTorchMLP(torch.nn.Module):  # One hidden layer
    def __init__(self, n_hidden=10, epochs=100, eta=0.001, minibatch_size=
        super().__init__()
        self.random = np.random.RandomState(seed)  # shuffle mini batches
        self.n_hidden = n_hidden  # size of the hidden layer
        self.epochs = epochs  # number of iterations
        self.eta = eta  # learning rate
        self.minibatch_size = minibatch_size  # size of training batch - 1
        self.optimizer = None
        self.loss_func = torch.nn.CrossEntropyLoss()
        self.model = None

    def init_layers(self, _M:int, _K:int) -> None:
        # data structure
        self.model = torch.nn.Sequential(
            torch.nn.Linear(_M, self.n_hidden),
            torch.nn.Sigmoid(),
            torch.nn.Linear(self.n_hidden, self.n_hidden),
            torch.nn.Sigmoid(),
            torch.nn.Linear(self.n_hidden, _K),
```

```
        )

    def predict(self, _X):
        _X = torch.FloatTensor(_X)
        assert self.model is not None
        self.model.eval()
        with torch.no_grad():
            y_pred = np.argmax(self.model(_X), axis=1)
        self.model.train()
        return y_pred.numpy()

    def fit(self, _X_train, _y_train, info=False):
        import sys
        _X_train, _y_train = torch.FloatTensor(_X_train), torch.LongTensor
        n_features= _X_train.shape[1]
        n_output= np.unique(_y_train).shape[0]  # number of class labels

        self.init_layers(n_features, n_output)
        self.optimizer = torch.optim.Rprop(self.model.parameters(), lr=sel

        for i in range(self.epochs):
            indices = np.arange(_X_train.shape[0])
            self.random.shuffle(indices)  # shuffle the data each epoch

            for start_idx in range(0, indices.shape[0] - self.minibatch_si
                batch_idx = indices[start_idx:start_idx + self.minibatch_s
                self.optimizer.zero_grad()

                net_out = self.model(_X_train[batch_idx])

                loss = self.loss_func(net_out, _y_train[batch_idx])
                loss.backward()
                self.optimizer.step()

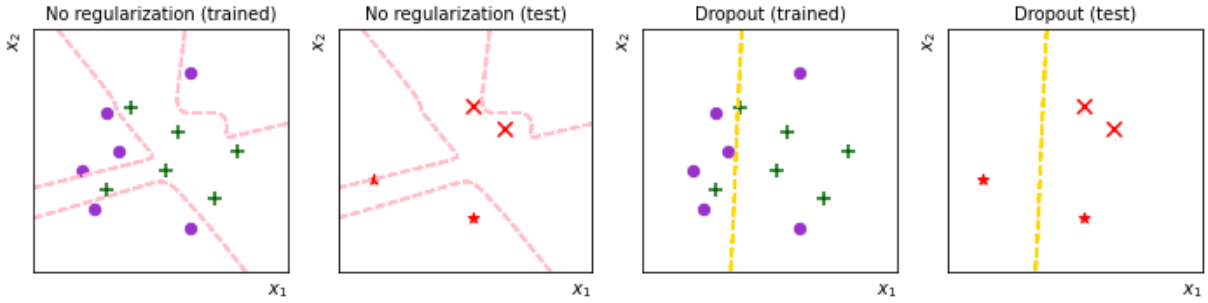
                if info:
                    sys.stderr.write(f"\r{i+1:03d} Loss: {loss.item():6.5f
                    sys.stderr.flush()
        return self
```

In [15]:
```
# A derived class to have Dropout
class MLP2(PyTorchMLP):
    def init_layers(self, _M, _K):
        self.model = torch.nn.Sequential(
            torch.nn.Linear(_M, self.n_hidden),
            torch.nn.Sigmoid(),
            torch.nn.Dropout(0.1),
            torch.nn.Linear(self.n_hidden, self.n_hidden),
            torch.nn.ReLU(),
            torch.nn.Dropout(0.1),
            torch.nn.Linear(self.n_hidden, _K),
        )
```

In [16]:
```
# Train
mlp1 = PyTorchMLP(n_hidden=10, epochs=100, eta=0.001, minibatch_size=X_tr_
```

```
mlp2 = MLP2(n_hidden=10, epochs=2000, eta=0.0001, minibatch_size=X_tr_sc.s
```

In [17]:
```
plotNN(X_tr_sc, y_tr, X_ts_sc, y_test, x1lim2, x2lim2, mlp1, mlp2, 'No reg
```




```
MLP 1 Reclassification Acc= 100.0%
MLP 1 Test Acc= 75.0%
MLP 2 Reclassification Acc= 75.0%
MLP 2 Test Acc= 75.0%
```

As can be seen above, dropout smooths out the decision boundary similar to the regularization.

References

1. Raschka, Sebastian, et al. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python. Packt Publishing Ltd, 2022.
2. Chintala, Soumith. "Deep learning with pytorch: A 60 minute blitz." (2017).

Exercises

Exercise 1. In SVM decision surface comparison, keep the gamma at 0.25 and vary C as in (0.1, 10, 1000). Observe the classifier model and report it.

Exercise 2. Change the SVM classifier to the random forest classifier, set its `n_estimators` to 5, vary the parameter `max_depth` as in (1,2,3) and plot the comparison decision surfaces as in SVM models comparison.

Exercise 3. In the code, explain how the decision boundary of the linear SVM is plotted. What is another name for it?

Exercise 4. In the code, explain how the decision boundary of the RBF SVM is plotted.

Exercise 5. In the PyTorch code, increase the dropout to 0.5 and comment about the results.

In [1]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```
